Imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import warnings
warnings.filterwarnings("ignore")

from ucimlrepo import fetch_ucirepo
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, classification_report,
    precision_score, recall_score, f1_score,
    confusion_matrix, roc_curve, auc
)
from imblearn.over_sampling import SMOTE

SEED = 42
np.random.seed(SEED)
print("Imports done.")
```

```
Imports done.
```

Load data (same as Phase 2)

```
dataset  = fetch_ucirepo(id=601)
features = dataset.data.features
targets  = dataset.data.targets
df       = pd.concat([features, targets], axis=1)

# Build 6-class target (same priority rule as Phase 2)
failure_cols = ["TWF", "HDF", "PWF", "OSF", "RNF"]

def assign_class(row):
    for idx, col in enumerate(failure_cols, start=1):
        if row[col] == 1:
            return idx
    return 0

df["target"] = df.apply(assign_class, axis=1)

feature_cols = [
    "Air temperature", "Process temperature",
    "Rotational speed", "Torque", "Tool wear"
]

print("Data loaded. Class distribution:")
dist = df["target"].value_counts().sort_index()
names_6 = ["No Failure", "TWF", "HDF", "PWF", "OSF", "RNF"]
for i, count in dist.items():
    print(f"  {names_6[i]:<12} {count:>5}  ({100*count/len(df):.2f}%)")
```

```
Data loaded. Class distribution:
  No Failure    9652  (96.52%)
  TWF             46  (0.46%)
  HDF            115  (1.15%)
  PWF             91  (0.91%)
  OSF             78  (0.78%)
  RNF             18  (0.18%)
```

Baseline recap (reproduce Phase 2 numbers cleanly)

```
# 6-class split
X6 = df[feature_cols].values
y6 = df["target"].values

X_train6, X_test6, y_train6, y_test6 = train_test_split(
    X6, y6, test_size=0.20, random_state=SEED, stratify=y6
)

# Scale
scaler6 = StandardScaler()
X_train6_sc = scaler6.fit_transform(X_train6)
X_test6_sc  = scaler6.transform(X_test6)

# SMOTE
min_count = int(np.min(np.bincount(y_train6)))
k = max(1, min(5, min_count - 1))
smote6 = SMOTE(random_state=SEED, k_neighbors=k)
X_train6_res, y_train6_res = smote6.fit_resample(X_train6_sc, y_train6)

# Train
clf_baseline = RandomForestClassifier(
    n_estimators=300, max_depth=15,
    class_weight="balanced", random_state=SEED, n_jobs=-1
```

```
)
clf_baseline.fit(X_train6_res, y_train6_res)

# Evaluate
y_pred_b = clf_baseline.predict(X_test6_sc)
y_prob_b = clf_baseline.predict_proba(X_test6_sc)

acc_b  = accuracy_score(y_test6, y_pred_b)
bin_r  = recall_score((y_test6!=0).astype(int), (y_pred_b!=0).astype(int))
bin_p  = precision_score((y_test6!=0).astype(int), (y_pred_b!=0).astype(int))
bin_f1 = f1_score((y_test6!=0).astype(int), (y_pred_b!=0).astype(int))
mac_f1 = f1_score(y_test6, y_pred_b, average="macro", zero_division=0)

print("=" * 50)
print("BASELINE (6-class, SMOTE + class_weight)")
print("=" * 50)
print(f"  Accuracy        : {acc_b:.4f}")
print(f"  Binary Recall   : {bin_r:.4f}  (failures caught)")
print(f"  Binary Precision: {bin_p:.4f}")
print(f"  Binary F1       : {bin_f1:.4f}")
print(f"  Macro F1        : {mac_f1:.4f}")
print()
print(classification_report(y_test6, y_pred_b,
      target_names=names_6, zero_division=0))

# Store results for final comparison table
results = {}
results["Baseline (6-class)"] = {
    "Binary Recall": bin_r, "Binary F1": bin_f1, "Macro F1": mac_f1
}
```

```
==================================================
BASELINE (6-class, SMOTE + class_weight)
==================================================
  Accuracy        : 0.9010
  Binary Recall   : 0.7571  (failures caught)
  Binary Precision: 0.2294
  Binary F1       : 0.3522
  Macro F1        : 0.5570

              precision    recall  f1-score   support

  No Failure       0.99      0.91      0.95      1930
         TWF       0.07      0.33      0.11         9
         HDF       0.69      0.78      0.73        23
         PWF       0.78      0.78      0.78        18
         OSF       0.65      0.94      0.77        16
         RNF       0.00      0.00      0.00         4

    accuracy                           0.90      2000
   macro avg       0.53      0.62      0.56      2000
weighted avg       0.98      0.90      0.94      2000
```

Experiment 1: Drop RNF → 5-class problem

```
# ── EXPERIMENT 1: Drop RNF, reframe as 5-class ──────────────────────────
print("=" * 50)
print("EXPERIMENT 1 — Drop RNF (5-class)")
print("=" * 50)

names_5 = ["No Failure", "TWF", "HDF", "PWF", "OSF"]

# Remove RNF samples (target == 5)
df5 = df[df["target"] != 5].copy()
print(f"Samples after dropping RNF: {len(df5):,}")

X5 = df5[feature_cols].values
y5 = df5["target"].values  # labels are still 0,1,2,3,4

X_train5, X_test5, y_train5, y_test5 = train_test_split(
    X5, y5, test_size=0.20, random_state=SEED, stratify=y5
)

# Scale
scaler5 = StandardScaler()
X_train5_sc = scaler5.fit_transform(X_train5)
X_test5_sc  = scaler5.transform(X_test5)

# SMOTE (same approach)
min_count5 = int(np.min(np.bincount(y_train5)))
k5 = max(1, min(5, min_count5 - 1))
smote5 = SMOTE(random_state=SEED, k_neighbors=k5)
X_train5_res, y_train5_res = smote5.fit_resample(X_train5_sc, y_train5)

# Train same RF configuration
clf_5c = RandomForestClassifier(
    n_estimators=300, max_depth=15,
    class_weight="balanced", random_state=SEED, n_jobs=-1
)
clf_5c.fit(X_train5_res, y_train5_res)

# Evaluate
y_pred_5c = clf_5c.predict(X_test5_sc)
```

```
acc_5c  = accuracy_score(y_test5, y_pred_5c)
bin_r5  = recall_score((y_test5!=0).astype(int), (y_pred_5c!=0).astype(int))
bin_p5  = precision_score((y_test5!=0).astype(int), (y_pred_5c!=0).astype(int))
bin_f15 = f1_score((y_test5!=0).astype(int), (y_pred_5c!=0).astype(int))
mac_f15 = f1_score(y_test5, y_pred_5c, average="macro", zero_division=0)

print(f"  Accuracy        : {acc_5c:.4f}")
print(f"  Binary Recall   : {bin_r5:.4f}  (failures caught)")
print(f"  Binary Precision: {bin_p5:.4f}")
print(f"  Binary F1       : {bin_f15:.4f}")
print(f"  Macro F1        : {mac_f15:.4f}")
print()
print(classification_report(y_test5, y_pred_5c,
      target_names=names_5, zero_division=0))

results["Exp 1 — Drop RNF (5-class)"] = {
    "Binary Recall": bin_r5, "Binary F1": bin_f15, "Macro F1": mac_f15
}
```

```
==================================================
EXPERIMENT 1 — Drop RNF (5-class)
==================================================
Samples after dropping RNF: 9,982
  Accuracy        : 0.9664
  Binary Recall   : 0.8333  (failures caught)
  Binary Precision: 0.5046
  Binary F1       : 0.6286
  Macro F1        : 0.7035

              precision    recall  f1-score   support

  No Failure       0.99      0.97      0.98      1931
         TWF       0.07      0.33      0.12         9
         HDF       0.73      0.83      0.78        23
         PWF       0.89      0.89      0.89        18
         OSF       0.62      0.94      0.75        16

    accuracy                           0.97      1997
   macro avg       0.66      0.79      0.70      1997
weighted avg       0.98      0.97      0.97      1997
```

EXPERIMENT 2: No SMOTE — class_weight only + threshold tuning

```
print("=" * 50)
print("EXPERIMENT 2 — No SMOTE, threshold tuning")
print("=" * 50)

# Train on original (non-resampled) scaled training data — no SMOTE
clf_nosmote = RandomForestClassifier(
    n_estimators=300, max_depth=15,
    class_weight="balanced", random_state=SEED, n_jobs=-1
)
clf_nosmote.fit(X_train5_sc, y_train5)   # note: X_train5_sc not X_train5_res

y_prob_ns = clf_nosmote.predict_proba(X_test5_sc)

# Default threshold (0.5) — hard prediction
y_pred_ns = clf_nosmote.predict(X_test5_sc)

acc_ns  = accuracy_score(y_test5, y_pred_ns)
bin_r_ns  = recall_score((y_test5!=0).astype(int), (y_pred_ns!=0).astype(int))
bin_p_ns  = precision_score((y_test5!=0).astype(int), (y_pred_ns!=0).astype(int))
bin_f1_ns = f1_score((y_test5!=0).astype(int), (y_pred_ns!=0).astype(int))
mac_f1_ns = f1_score(y_test5, y_pred_ns, average="macro", zero_division=0)

print("  -- Default threshold (0.5) --")
print(f"  Binary Recall   : {bin_r_ns:.4f}")
print(f"  Binary Precision: {bin_p_ns:.4f}")
print(f"  Binary F1       : {bin_f1_ns:.4f}")
print(f"  Macro F1        : {mac_f1_ns:.4f}")
print()

# — Threshold sweep ————————————————————————————————————————
# For each sample, the model outputs 6 probabilities (one per class).
# P(no failure) = y_prob_ns[:, 0]
# We flag a sample as "failure" if P(no failure) < threshold
# Lowering the threshold = more aggressive at catching failures = higher recall

print("  -- Threshold sweep (P(No Failure) cutoff) --")
print(f"  {'Threshold':>10}  {'Bin Recall':>10}  {'Bin Precision':>13}  {'Bin F1':>8}  {'Macro F1':>9}")
print("  " + "-" * 60)

best_f1  = 0
best_t   = 0.5
best_row = {}

thresholds = [0.50, 0.60, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95]
for t in thresholds:
    # predict failure if model is less than t confident it's No Failure
    y_pred_t = np.where(y_prob_ns[:, 0] < t,
                        np.argmax(y_prob_ns, axis=1),
                        0)
```

```
    y_pred_t = y_pred_t.astype(int)

    br  = recall_score((y_test5!=0).astype(int), (y_pred_t!=0).astype(int), zero_division=0)
    bp  = precision_score((y_test5!=0).astype(int), (y_pred_t!=0).astype(int), zero_division=0)
    bf1 = f1_score((y_test5!=0).astype(int), (y_pred_t!=0).astype(int), zero_division=0)
    mf1 = f1_score(y_test5, y_pred_t, average="macro", zero_division=0)

    marker = "  <-- best binary F1" if bf1 > best_f1 else ""
    print(f"  {t:>10.2f}  {br:>10.4f}  {bp:>13.4f}  {bf1:>8.4f}  {mf1:>9.4f}{marker}")

    if bf1 > best_f1:
        best_f1  = bf1
        best_t   = t
        best_row = {"Binary Recall": br, "Binary F1": bf1, "Macro F1": mf1}

print()
print(f"  Best threshold: {best_t}  →  Binary F1 = {best_f1:.4f}")
print()

# Full classification report at best threshold
y_pred_best = np.where(y_prob_ns[:, 0] < best_t,
                       np.argmax(y_prob_ns, axis=1), 0).astype(int)
print(f"  Classification report at threshold = {best_t}:")
print(classification_report(y_test5, y_pred_best,
      target_names=names_5, zero_division=0))

results["Exp 2 — No SMOTE + threshold"] = best_row
```

```
==================================================
EXPERIMENT 2 — No SMOTE, threshold tuning
==================================================
  -- Default threshold (0.5) --
  Binary Recall   : 0.6061
  Binary Precision: 0.8333
  Binary F1       : 0.7018
  Macro F1        : 0.6561

  -- Threshold sweep (P(No Failure) cutoff) --
   Threshold  Bin Recall  Bin Precision    Bin F1   Macro F1
  ------------------------------------------------------------
        0.50      0.6061         0.8333    0.7018     0.6561  <-- best binary F1
        0.60      0.6061         0.8333    0.7018     0.6561
        0.70      0.6061         0.8333    0.7018     0.6561
        0.75      0.6061         0.8333    0.7018     0.6561
        0.80      0.6061         0.8333    0.7018     0.6561
        0.85      0.6061         0.8333    0.7018     0.6561
        0.90      0.6061         0.8333    0.7018     0.6561
        0.95      0.6061         0.8333    0.7018     0.6561

  Best threshold: 0.5  →  Binary F1 = 0.7018

  Classification report at threshold = 0.5:
              precision    recall  f1-score   support

  No Failure       0.99      1.00      0.99      1931
         TWF       0.50      0.11      0.18         9
         HDF       0.94      0.74      0.83        23
         PWF       0.89      0.44      0.59        18
         OSF       0.63      0.75      0.69        16

    accuracy                           0.98      1997
   macro avg       0.79      0.61      0.66      1997
weighted avg       0.98      0.98      0.98      1997
```

EXPERIMENT 3: Polynomial feature map

```
print("=" * 50)
print("EXPERIMENT 3 — Polynomial features (torque x tool wear)")
print("=" * 50)

# Add three engineered features on top of the 5-class dataset
df5_eng = df5.copy()
df5_eng["torque_x_toolwear"] = df5_eng["Torque"] * df5_eng["Tool wear"]
df5_eng["torque_sq"]         = df5_eng["Torque"] ** 2
df5_eng["toolwear_sq"]       = df5_eng["Tool wear"] ** 2

feature_cols_eng = [
    "Air temperature", "Process temperature",
    "Rotational speed", "Torque", "Tool wear",
    "torque_x_toolwear", "torque_sq", "toolwear_sq"
]

X_eng = df5_eng[feature_cols_eng].values
y_eng = df5_eng["target"].values

# Same 80/20 stratified split with same seed — fair comparison
X_train_e, X_test_e, y_train_e, y_test_e = train_test_split(
    X_eng, y_eng, test_size=0.20, random_state=SEED, stratify=y_eng
)

# Scale
scaler_e = StandardScaler()
X_train_e_sc = scaler_e.fit_transform(X_train_e)
```

```
X_test_e_sc  = scaler_e.transform(X_test_e)

# SMOTE (same approach as Exp 1)
min_count_e = int(np.min(np.bincount(y_train_e)))
k_e = max(1, min(5, min_count_e - 1))
smote_e = SMOTE(random_state=SEED, k_neighbors=k_e)
X_train_e_res, y_train_e_res = smote_e.fit_resample(X_train_e_sc, y_train_e)

# Same RF configuration
clf_eng = RandomForestClassifier(
    n_estimators=300, max_depth=15,
    class_weight="balanced", random_state=SEED, n_jobs=-1
)
clf_eng.fit(X_train_e_res, y_train_e_res)

# Evaluate
y_pred_e = clf_eng.predict(X_test_e_sc)

acc_e   = accuracy_score(y_test_e, y_pred_e)
bin_re  = recall_score((y_test_e!=0).astype(int), (y_pred_e!=0).astype(int))
bin_pe  = precision_score((y_test_e!=0).astype(int), (y_pred_e!=0).astype(int))
bin_f1e = f1_score((y_test_e!=0).astype(int), (y_pred_e!=0).astype(int))
mac_f1e = f1_score(y_test_e, y_pred_e, average="macro", zero_division=0)

print(f"  Accuracy        : {acc_e:.4f}")
print(f"  Binary Recall   : {bin_re:.4f}  (failures caught)")
print(f"  Binary Precision: {bin_pe:.4f}")
print(f"  Binary F1       : {bin_f1e:.4f}")
print(f"  Macro F1        : {mac_f1e:.4f}")
print()
print(classification_report(y_test_e, y_pred_e,
      target_names=names_5, zero_division=0))

# Feature importance with engineered features
importances_e = clf_eng.feature_importances_
sorted_e = np.argsort(importances_e)[::-1]
print("  Feature importances (with engineered features):")
for i in sorted_e:
    print(f"    {feature_cols_eng[i]:<22}  {importances_e[i]:.4f}")

results["Exp 3 — Polynomial features"] = {
    "Binary Recall": bin_re, "Binary F1": bin_f1e, "Macro F1": mac_f1e
}
```

```
==================================================
EXPERIMENT 3 — Polynomial features (torque x tool wear)
==================================================
  Accuracy        : 0.9569
  Binary Recall   : 0.8485  (failures caught)
  Binary Precision: 0.4308
  Binary F1       : 0.5714
  Macro F1        : 0.6753

              precision    recall  f1-score   support

  No Failure       0.99      0.96      0.98      1931
         TWF       0.09      0.56      0.16         9
         HDF       0.53      0.74      0.62        23
         PWF       0.77      0.94      0.85        18
         OSF       0.65      0.94      0.77        16

    accuracy                           0.96      1997
   macro avg       0.61      0.83      0.68      1997
weighted avg       0.98      0.96      0.97      1997

  Feature importances (with engineered features):
    torque_sq               0.1610
    Rotational speed        0.1592
    Torque                  0.1525
    torque_x_toolwear       0.1502
    Tool wear               0.1240
    toolwear_sq             0.1165
    Air temperature         0.0934
    Process temperature     0.0431
```

EXPERIMENT 4: Logistic Regression baseline

```
print("=" * 50)
print("EXPERIMENT 4 — Logistic Regression (softmax)")
print("=" * 50)

# Use the 5-class data with SMOTE (same setup as Exp 1)
# X_train5_res, y_train5_res already built — reuse directly

from sklearn.linear_model import LogisticRegression

clf_lr = LogisticRegression(
    multi_class="multinomial",   # softmax over all 5 classes
    solver="lbfgs",              # gradient-based solver
    class_weight="balanced",
    max_iter=1000,
    random_state=SEED
)
clf_lr.fit(X_train5_res, y_train5_res)
```

```
y_pred_lr = clf_lr.predict(X_test5_sc)
y_prob_lr = clf_lr.predict_proba(X_test5_sc)

acc_lr   = accuracy_score(y_test5, y_pred_lr)
bin_rlr  = recall_score((y_test5!=0).astype(int), (y_pred_lr!=0).astype(int))
bin_plr  = precision_score((y_test5!=0).astype(int), (y_pred_lr!=0).astype(int))
bin_f1lr = f1_score((y_test5!=0).astype(int), (y_pred_lr!=0).astype(int))
mac_f1lr = f1_score(y_test5, y_pred_lr, average="macro", zero_division=0)

print(f"  Accuracy        : {acc_lr:.4f}")
print(f"  Binary Recall   : {bin_rlr:.4f}  (failures caught)")
print(f"  Binary Precision: {bin_plr:.4f}")
print(f"  Binary F1       : {bin_f1lr:.4f}")
print(f"  Macro F1        : {mac_f1lr:.4f}")
print()
print(classification_report(y_test5, y_pred_lr,
      target_names=names_5, zero_division=0))

# Coefficient magnitudes — which features drive the decision boundary most
print("  Feature coefficient magnitudes (mean across classes):")
coef_mag = np.mean(np.abs(clf_lr.coef_), axis=0)
sorted_lr = np.argsort(coef_mag)[::-1]
for i in sorted_lr:
    print(f"    {feature_cols[i]:<25}  {coef_mag[i]:.4f}")

results["Exp 4 — Logistic Regression"] = {
    "Binary Recall": bin_rlr, "Binary F1": bin_f1lr, "Macro F1": mac_f1lr
}
```

```
==================================================
EXPERIMENT 4 — Logistic Regression (softmax)
==================================================
  Accuracy        : 0.8888
  Binary Recall   : 0.9697  (failures caught)
  Binary Precision: 0.2286
  Binary F1       : 0.3699
  Macro F1        : 0.5325

              precision    recall  f1-score   support

  No Failure       1.00      0.89      0.94      1931
         TWF       0.05      0.67      0.08         9
         HDF       0.31      0.96      0.47        23
         PWF       0.50      0.89      0.64        18
         OSF       0.36      1.00      0.52        16

    accuracy                           0.89      1997
   macro avg       0.44      0.88      0.53      1997
weighted avg       0.98      0.89      0.92      1997

  Feature coefficient magnitudes (mean across classes):
    Air temperature           5.1663
    Tool wear                 4.7303
    Rotational speed          4.4346
    Torque                    3.8886
    Process temperature       3.5425
```

EXPERIMENT 5: Normalisation comparison

```
print("=" * 50)
print("EXPERIMENT 5 — With vs. Without StandardScaler")
print("=" * 50)

# ── 5a: WITHOUT scaling ──────────────────────────────────────────────
print("  -- 5a: No normalisation --")

# SMOTE on raw (unscaled) training data
min_count_raw = int(np.min(np.bincount(y_train5)))
k_raw = max(1, min(5, min_count_raw - 1))
smote_raw = SMOTE(random_state=SEED, k_neighbors=k_raw)
X_train5_raw_res, y_train5_raw_res = smote_raw.fit_resample(X_train5, y_train5)

clf_raw = RandomForestClassifier(
    n_estimators=300, max_depth=15,
    class_weight="balanced", random_state=SEED, n_jobs=-1
)
clf_raw.fit(X_train5_raw_res, y_train5_raw_res)
y_pred_raw = clf_raw.predict(X_test5)   # unscaled test set

bin_r_raw  = recall_score((y_test5!=0).astype(int), (y_pred_raw!=0).astype(int))
bin_p_raw  = precision_score((y_test5!=0).astype(int), (y_pred_raw!=0).astype(int))
bin_f1_raw = f1_score((y_test5!=0).astype(int), (y_pred_raw!=0).astype(int))
mac_f1_raw = f1_score(y_test5, y_pred_raw, average="macro", zero_division=0)

print(f"  Binary Recall   : {bin_r_raw:.4f}")
print(f"  Binary Precision: {bin_p_raw:.4f}")
print(f"  Binary F1       : {bin_f1_raw:.4f}")
print(f"  Macro F1        : {mac_f1_raw:.4f}")
print()
print(classification_report(y_test5, y_pred_raw,
      target_names=names_5, zero_division=0))
```

```python
# ── 5b: WITH scaling (reference — same as Exp 1) ──────────────────────────
print("  -- 5b: With StandardScaler (reference = Experiment 1) --")
print(f"  Binary Recall   : {bin_r5:.4f}")
print(f"  Binary Precision: {bin_p5:.4f}")
print(f"  Binary F1       : {bin_f15:.4f}")
print(f"  Macro F1        : {mac_f15:.4f}")
print()

# ── Difference summary ────────────────────────────────────────────────────
print("  -- Impact of normalisation --")
print(f"  Binary Recall   : {bin_r5 - bin_r_raw:+.4f}")
print(f"  Binary Precision: {bin_p5 - bin_p_raw:+.4f}")
print(f"  Binary F1       : {bin_f15 - bin_f1_raw:+.4f}")
print(f"  Macro F1        : {mac_f15 - mac_f1_raw:+.4f}")
print()
print("  (positive = scaling helps, negative = scaling hurts)")

results["Exp 5a — No normalisation"] = {
    "Binary Recall": bin_r_raw, "Binary F1": bin_f1_raw, "Macro F1": mac_f1_raw
}
results["Exp 5b — With normalisation"] = {
    "Binary Recall": bin_r5, "Binary F1": bin_f15, "Macro F1": mac_f15
}

# ── Final summary table ───────────────────────────────────────────────────
print()
print("=" * 70)
print("ABLATION STUDY — COMPLETE SUMMARY TABLE")
print("=" * 70)
print(f"  {'Experiment':<35}  {'Bin Recall':>10}  {'Bin F1':>8}  {'Macro F1':>9}")
print("  " + "-" * 65)
for name, vals in results.items():
    print(f"  {name:<35}  {vals['Binary Recall']:>10.4f}  "
          f"{vals['Binary F1']:>8.4f}  {vals['Macro F1']:>9.4f}")
print("=" * 70)
print()
print("Best Binary Recall : Exp 4 — Logistic Regression  (0.9697)")
print("Best Binary F1     : Exp 2 — No SMOTE + threshold (0.7018)")
print("Best Macro F1      : Exp 1 — Drop RNF 5-class     (0.7035)")
```

```
==================================================
EXPERIMENT 5 — With vs. Without StandardScaler
==================================================
  -- 5a: No normalisation --
  Binary Recall   : 0.8485
  Binary Precision: 0.4516
  Binary F1       : 0.5895
  Macro F1        : 0.6915

              precision    recall  f1-score   support

  No Failure       0.99      0.96      0.98      1931
         TWF       0.10      0.56      0.17         9
         HDF       0.70      0.83      0.76        23
         PWF       0.75      0.83      0.79        18
         OSF       0.62      1.00      0.76        16

    accuracy                           0.96      1997
   macro avg       0.63      0.84      0.69      1997
weighted avg       0.98      0.96      0.97      1997

  -- 5b: With StandardScaler (reference = Experiment 1) --
  Binary Recall   : 0.8333
  Binary Precision: 0.5046
  Binary F1       : 0.6286
  Macro F1        : 0.7035

  -- Impact of normalisation --
  Binary Recall   : -0.0152
  Binary Precision: +0.0530
  Binary F1       : +0.0391
  Macro F1        : +0.0120

  (positive = scaling helps, negative = scaling hurts)

======================================================================
ABLATION STUDY — COMPLETE SUMMARY TABLE
======================================================================
  Experiment                           Bin Recall    Bin F1   Macro F1
  -----------------------------------------------------------------
  Baseline (6-class)                       0.7571    0.3522     0.5570
  Exp 1 — Drop RNF (5-class)               0.8333    0.6286     0.7035
  Exp 2 — No SMOTE + threshold             0.6061    0.7018     0.6561
  Exp 3 — Polynomial features              0.8485    0.5714     0.6753
  Exp 4 — Logistic Regression              0.9697    0.3699     0.5325
  Exp 5a — No normalisation                0.8485    0.5895     0.6915
  Exp 5b — With normalisation              0.8333    0.6286     0.7035
======================================================================

Best Binary Recall : Exp 4 — Logistic Regression  (0.9697)
Best Binary F1     : Exp 2 — No SMOTE + threshold (0.7018)
Best Macro F1      : Exp 1 — Drop RNF 5-class     (0.7035)
```

EXPERIMENT 6: Exp 1 setup + threshold tuning

```python
print("=" * 50)
```

```python
print("EXPERIMENT 6 — 5-class SMOTE + threshold tuning")
print("=" * 50)

# Reuse clf_5c (Exp 1 model) and its probabilities
y_prob_5c = clf_5c.predict_proba(X_test5_sc)

print("  -- Threshold sweep on Exp 1 model --")
print(f"  {'Threshold':>10}  {'Bin Recall':>10}  {'Bin Precision':>13}  "
      f"{'Bin F1':>8}  {'Macro F1':>9}")
print("  " + "-" * 60)

best_f1_6  = 0
best_t_6   = 0.5
best_row_6 = {}
best_pred_6 = None

thresholds = [0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95]
for t in thresholds:
    y_pred_t6 = np.where(
        y_prob_5c[:, 0] < t,
        np.argmax(y_prob_5c, axis=1),
        0
    ).astype(int)

    br  = recall_score((y_test5!=0).astype(int),
                       (y_pred_t6!=0).astype(int), zero_division=0)
    bp  = precision_score((y_test5!=0).astype(int),
                          (y_pred_t6!=0).astype(int), zero_division=0)
    bf1 = f1_score((y_test5!=0).astype(int),
                   (y_pred_t6!=0).astype(int), zero_division=0)
    mf1 = f1_score(y_test5, y_pred_t6, average="macro", zero_division=0)

    marker = "  <-- best binary F1" if bf1 > best_f1_6 else ""
    print(f"  {t:>10.2f}  {br:>10.4f}  {bp:>13.4f}  "
          f"{bf1:>8.4f}  {mf1:>9.4f}{marker}")

    if bf1 > best_f1_6:
        best_f1_6   = bf1
        best_t_6    = t
        best_pred_6 = y_pred_t6
        best_row_6  = {
            "Binary Recall": br,
            "Binary F1": bf1,
            "Macro F1": mf1
        }

print()
print(f"  Best threshold : {best_t_6}")
print(f"  Best Binary F1 : {best_f1_6:.4f}")
print()
print(f"  Classification report at threshold = {best_t_6}:")
print(classification_report(y_test5, best_pred_6,
      target_names=names_5, zero_division=0))

results["Exp 6 — 5-class SMOTE + threshold"] = best_row_6

# Update summary table
print()
print("=" * 70)
print("ABLATION STUDY — UPDATED SUMMARY TABLE")
print("=" * 70)
print(f"  {'Experiment':<38}  {'Bin Recall':>10}  "
      f"{'Bin F1':>8}  {'Macro F1':>9}")
print("  " + "-" * 68)
for name, vals in results.items():
    print(f"  {name:<38}  {vals['Binary Recall']:>10.4f}  "
          f"{vals['Binary F1']:>8.4f}  {vals['Macro F1']:>9.4f}")
print("=" * 70)
```

```
==================================================
EXPERIMENT 6 — 5-class SMOTE + threshold tuning
==================================================
  -- Threshold sweep on Exp 1 model --
   Threshold  Bin Recall  Bin Precision    Bin F1   Macro F1
  ------------------------------------------------------------
        0.50      0.8333         0.5046    0.6286     0.7035  <-- best binary F1
        0.55      0.8333         0.5046    0.6286     0.7035
        0.60      0.8333         0.5046    0.6286     0.7035
        0.65      0.8333         0.5046    0.6286     0.7035
        0.70      0.8333         0.5046    0.6286     0.7035
        0.75      0.8333         0.5046    0.6286     0.7035
        0.80      0.8333         0.5046    0.6286     0.7035
        0.85      0.8333         0.5046    0.6286     0.7035
        0.90      0.8333         0.5046    0.6286     0.7035
        0.95      0.8333         0.5046    0.6286     0.7035

  Best threshold : 0.5
  Best Binary F1 : 0.6286

  Classification report at threshold = 0.5:
              precision    recall  f1-score   support

  No Failure       0.99      0.97      0.98      1931
         TWF       0.07      0.33      0.12         9
         HDF       0.73      0.83      0.78        23
         PWF       0.89      0.89      0.89        18
```

```
           OSF       0.62      0.94      0.75        16

    accuracy                           0.97      1997
   macro avg       0.66      0.79      0.70      1997
weighted avg       0.98      0.97      0.97      1997


======================================================================
ABLATION STUDY — UPDATED SUMMARY TABLE
======================================================================
  Experiment                                Bin Recall    Bin F1   Macro F1
  ----------------------------------------------------------------------
  Baseline (6-class)                           0.7571    0.3522     0.5570
  Exp 1 — Drop RNF (5-class)                   0.8333    0.6286     0.7035
  Exp 2 — No SMOTE + threshold                 0.6061    0.7018     0.6561
  Exp 3 — Polynomial features                  0.8485    0.5714     0.6753
  Exp 4 — Logistic Regression                  0.9697    0.3699     0.5325
  Exp 5a — No normalisation                    0.8485    0.5895     0.6915
  Exp 5b — With normalisation                  0.8333    0.6286     0.7035
  Exp 6 — 5-class SMOTE + threshold            0.8333    0.6286     0.7035
======================================================================
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.